

18-Month Master Learning Roadmap

Full Stack Engineer with Mobile & Cloud Focus

Your Complete Tech Stack (9 Skills)

- 1.  **Python** (Advanced)
 - 2.  **Java + Spring Boot** (Advanced)
 - 3.  **React** (Advanced Web Frontend)
 - 4.  **Flutter** (Advanced Mobile - iOS & Android)
 - 5.  **Go** (Intermediate Microservices)
 - 6.  **AWS** (Advanced Cloud Infrastructure)
 - 7.  **Docker + Kubernetes** (Advanced DevOps)
 - 8.  **AI/ML** (Intermediate)
 - 9.  **Trading** (Personal learning - evenings/weekends separate time)
-

Learning Capacity

Metric	Value
Weekdays	3-4 hrs/day (increased from 2)
Weekends	4 hrs/day (Saturday) + 4 hrs (Sunday)
Total Hours/Week	~42-45 hours
Total Hours/18 months	~1100-1200 hours
Learning Style	Solo learner, project-based
Budget	Free + Udemy subscription
OS	Mac

Target Role & Salary

- **Position:** Full Stack Engineer (React + Flutter + Spring Boot)
- **Companies:** Startups, tech companies, fintech platforms

- **Realistic Salary:** \$120K-\$180K+ (depending on location & market)
-
-



THE 18-MONTH PLAN



PHASE 1: Foundations & Backend (Months 1-3)

Goal: Strengthen Python, learn Java/Spring Boot, setup everything

Month 1 — Python Advanced + Java Basics

Weekdays (3-4 hrs/day)

- **Week 1-2: Python Advanced**
 - Decorators, generators, context managers, async/await
 - Magic methods, dataclasses, abstract classes
 - Design patterns (Singleton, Factory, Observer)
- **Week 3-4: Java Fundamentals**
 - Syntax, OOP concepts, collections (List, Set, Map)
 - Streams API, lambda expressions, functional programming

Weekends (8 hrs/day)

- **Saturday:** Build Python CLI tool (budget tracker with OOP + decorators)
- **Sunday:** Learn Java syntax + build Java console app

Resources

- *Python Pro Bootcamp* — Jose Portilla (Udemy)
- *Java Masterclass* — Tim Buchalka (Udemy)
- Corey Schafer Python tutorials (YouTube - free)

Monthly Project

- Python: Budget tracker CLI with file I/O + decorators
- Java: Bank account system (OOP + collections)
- Both pushed to GitHub with proper READMEs

Deliverables

-  2 projects on GitHub
 -  Master decorators & generators in Python
 -  Comfortable with Java OOP
-

Month 2 — Spring Boot Intro + Docker

Weekdays (3-4 hrs/day)

- **Week 1-2:** Spring Boot Basics
 - Project setup, annotations, dependency injection
 - REST API creation, MVC pattern, request/response
- **Week 3-4:** Docker Fundamentals
 - Containers vs images, Dockerfile, Docker commands
 - Networking, volumes, best practices

Weekends (8 hrs/day)

- **Saturday:** Build Spring Boot REST API (Todo app CRUD)
- **Sunday:** Dockerize app + Docker Compose with database

Resources

- *Spring Boot 3 & Spring Framework 6* — Chad Darby (Udemy)
- *Docker & Kubernetes Complete Guide* — Stephen Grider (Udemy)

Monthly Project

- Full Stack Backend: Spring Boot REST API (Todo CRUD)
- Dockerized with PostgreSQL in docker-compose.yml
- Database migrations, proper error handling

Deliverables

-  REST API project on GitHub
 -  docker-compose.yml working locally
 -  API documentation (README + Postman collection)
-

Month 3 — Spring Boot Intermediate + Java Advanced

Weekdays (3-4 hrs/day)

- **Week 1-2:** Spring Boot Advanced
 - JPA/Hibernate, database integration, PostgreSQL
 - Exception handling, validation, logging
- **Week 3-4:** Java Advanced
 - Generics deep dive, concurrency basics
 - Testing with JUnit 5, Mockito

Weekends (8 hrs/day)

- **Saturday:** Add JPA/Hibernate to your Todo app + write tests
- **Sunday:** Improve Docker setup + add API documentation

Resources

- *Spring Boot + JPA/Hibernate* — Chad Darby (Udemy)
- Baeldung (free blog tutorials)

Monthly Project

- Enhanced Spring Boot app with:
 - Database layer (JPA/Hibernate)
 - Unit tests (80%+ coverage)
 - Proper exception handling
 - API documentation

Deliverables

-  Production-ready backend API
 -  Test suite with 80%+ coverage
 -  Database schema documentation
-

PHASE 2: React + Web Frontend (Months 4-6)

Goal: Learn React deeply, build full-stack web app

Month 4 — React Fundamentals

Weekdays (3-4 hrs/day)

- **Week 1:** HTML/CSS Crash Course
 - Semantic HTML, CSS flexbox, grid, responsive design
 - Mobile-first approach, media queries
- **Week 2-3:** JavaScript ES6+
 - Arrow functions, destructuring, spread operator
 - Promises, async/await, fetch API, JSON
- **Week 4:** React Basics
 - JSX, components, props, state with useState
 - React hooks basics, event handling

Weekends (8 hrs/day)

- **Saturday:** Build static portfolio site (HTML/CSS)
- **Sunday:** Build React Todo app with useState & localStorage

Resources

- *React — The Complete Guide 2024* — Maximilian Schwarzmüller (Udemy)
- *JavaScript: The Complete Guide* — Maximilian Schwarzmüller (Udemy)
- MDN Web Docs (free)

Monthly Project

- Portfolio website: HTML/CSS (responsive, mobile-friendly)
- React Todo app with local state management

Deliverables

-  Responsive portfolio deployed on GitHub Pages
 -  React app with proper component structure
 -  Clean, semantic code
-

Month 5 — React Intermediate + Full Stack Integration

Weekdays (3-4 hrs/day)

- **Week 1-2:** React Advanced

- React Router (navigation, URL params, nested routes)
- useEffect, useContext, custom hooks
- State management patterns
- **Week 3-4: Full Stack Integration**
 - Connecting React to Spring Boot backend
 - Authentication (JWT), error handling
 - API integration with axios/fetch

Weekends (8 hrs/day)

- **Saturday:** Build React Notes app with API integration
- **Sunday:** Add JWT authentication flow + error handling

Resources

- React docs (free - react.dev)
- Axios library docs (free)

Monthly Project

- Full Stack Notes App:
 - React frontend with routing & state management
 - Spring Boot REST API backend
 - JWT authentication
 - Dockerized both frontend & backend

Deliverables

-  Full-stack application on GitHub
 -  Docker setup for both services
 -  Authentication system working end-to-end
-

Month 6 — React Advanced + Performance Optimization

Weekdays (3-4 hrs/day)

- **Week 1-2: State Management & Advanced Patterns**
 - Redux Toolkit (or Context API advanced)
 - Managing complex state, async operations
 - Custom hooks patterns

- **Week 3-4: React Performance & Styling**
 - Code splitting, lazy loading, memoization
 - Tailwind CSS, component libraries (shadcn/ui)
 - Responsive design patterns

Weekends (8 hrs/day)

- **Saturday:** Implement Redux + optimize performance
- **Sunday:** Add Tailwind CSS styling + responsive design

Resources

- *Redux Toolkit* — Stephen Grider (Udemy)
- Tailwind CSS docs (free)
- React performance best practices (free blog)

Monthly Project

- Enhanced Notes App:
 - Redux Toolkit for state management
 - Tailwind CSS styling (beautiful UI)
 - Performance optimized (code splitting, lazy loading)
 - Responsive on mobile, tablet, desktop

Deliverables

-  Production-ready React application
-  Global state management working smoothly
-  Mobile-responsive design
-  Deployed on Vercel/Netlify (live link)

PHASE 3: Flutter Mobile Development (Months 7-9)

Goal: Master Flutter, build native iOS & Android apps

Month 7 — Flutter Fundamentals + Dart

Weekdays (3-4 hrs/day)

- **Week 1:** Dart Language Basics

- Syntax, types, null safety, functions
- Classes, inheritance, mixins
- Collections (List, Map, Set)
- **Week 2-4: Flutter Basics**
 - Setup (Xcode for Mac, Android Studio, VS Code)
 - Widgets, stateless vs stateful widgets
 - Hot reload, debugging, Flutter project structure

Weekends (8 hrs/day)

- **Saturday:** Flutter fundamentals + widget exploration
- **Sunday:** Build simple Flutter Todo app (UI only)

Resources

- *Flutter & Dart — The Complete Guide* — Maximilian Schwarzmüller (Udemy)
- Official Flutter docs (free - flutter.dev)
- Dart official tour (free)

Monthly Project

- Flutter Todo App (UI):
 - Stateless & stateful widgets
 - Basic navigation
 - Lists, forms, buttons

Deliverables

-  Flutter project on GitHub
-  App running on iOS simulator (on Mac)
-  App running on Android emulator

Month 8 — Flutter Intermediate + API Integration

Weekdays (3-4 hrs/day)

- **Week 1-2: Flutter Architecture & State Management**
 - Provider pattern for state management
 - Networking with Dio/http packages

- JSON parsing, error handling
- **Week 3-4: Flutter UI/UX**
 - Material Design implementation
 - Custom themes, animations
 - Navigation (named routes, arguments)

Weekends (8 hrs/day)

- **Saturday:** Add API integration to Flutter app
- **Sunday:** Implement state management with Provider

Resources

- Provider package docs (free)
- Dio HTTP client (free)
- Material Design guidelines (free)

Monthly Project

- Flutter Notes App (Full Featured):
 - Connect to Spring Boot backend API
 - Provider for state management
 - Create, read, update, delete notes
 - Proper error handling & loading states
 - Material Design UI

Deliverables

-  Full-featured Flutter app
 -  API integration working
 -  Running on both iOS simulator & Android emulator
 -  GitHub with proper documentation
-

Month 9 — Flutter Advanced + Cross-Platform Optimization

Weekdays (3-4 hrs/day)

- **Week 1-2: Advanced Flutter Features**
 - Local storage (Hive, SharedPreferences)
 - Background tasks, push notifications

- Platform channels (calling native code if needed)
- **Week 3-4: Platform-Specific Optimization**
 - iOS-specific considerations
 - Android-specific considerations
 - Testing (unit & widget tests)
 - Firebase integration (optional)

Weekends (8 hrs/day)

- **Saturday:** Add local storage + Firebase integration
- **Sunday:** Write tests + optimize for both platforms

Resources

- Hive database docs (free)
- Firebase docs (free)
- Flutter testing docs (free)

Monthly Project

- Production-Ready Flutter App:
 - Local storage with Hive
 - Push notifications
 - Offline support
 - Unit & widget tests (80%+ coverage)
 - Optimized for iOS & Android
 - Firebase backend (optional)

Deliverables

-  App ready for app stores
-  Proper error handling
-  Test coverage 80%+
-  iOS & Android optimization complete

PHASE 4: Cloud & DevOps (Months 10-12)

Goal: Master AWS, Kubernetes, CI/CD pipelines

Month 10 — AWS Fundamentals + Cloud Architecture

Weekdays (3-4 hrs/day)

- **Week 1-2:** AWS Core Services
 - EC2 (compute), S3 (storage), RDS (databases)
 - IAM (authentication & authorization)
 - VPC (networking), security groups
- **Week 3-4:** Cloud Architecture Basics
 - High availability, fault tolerance, scaling
 - Load balancing, auto-scaling concepts
 - Cost optimization

Weekends (8 hrs/day)

- **Saturday:** Deploy Spring Boot app to EC2
- **Sunday:** Set up RDS, S3 buckets, understand architecture

Resources

- *AWS Certified Cloud Practitioner* — Stephane Maarek (Udemy)
- AWS free tier (hands-on practice)
- AWS Well-Architected Framework (free whitepaper)

🏆 **Certification Target** AWS Cloud Practitioner — prepare this month

Monthly Project

- Deploy your full stack app (React + Spring Boot) to AWS:
 - Spring Boot on EC2
 - React on S3 + CloudFront
 - PostgreSQL on RDS
 - Proper security (IAM, security groups)

Deliverables

- ☒ App deployed and accessible on AWS
 - ☒ AWS Cloud Practitioner exam prep complete
 - ☒ Architecture documentation
-

Month 11 — AWS Advanced + Kubernetes

Weekdays (3-4 hrs/day)

- **Week 1-2:** AWS Advanced Services
 - Lambda (serverless), API Gateway
 - ECS (container orchestration)
 - CloudWatch (monitoring), SNS/SQS (messaging)
- **Week 3-4:** Kubernetes Fundamentals
 - Architecture (master, nodes, pods)
 - Deployments, services, ingress
 - StatefulSets, ConfigMaps, secrets
 - kubectl commands, debugging

Weekends (8 hrs/day)

- **Saturday:** Set up Kubernetes locally (minikube on Mac)
- **Sunday:** Deploy multi-container app to Kubernetes

Resources

- *Docker & Kubernetes Complete Guide* — Stephen Grider (Udemy)
- Kubernetes official docs (free)
- Play with Kubernetes (free sandbox)

🏆 **Certification Target** AWS Cloud Practitioner exam — take & pass this month ✅

Monthly Project

- Kubernetes Deployment:
 - React frontend in container
 - Spring Boot backend in container
 - PostgreSQL in container
 - All orchestrated with Kubernetes locally
 - Documentation & deployment files

Deliverables

- ✅ Kubernetes yaml files for your stack
- ✅ App running smoothly on local K8s
- ✅ Understanding of pods, services, deployments

-  AWS Cloud Practitioner certification earned 
-

Month 12 — CI/CD Pipelines + EKS

Weekdays (3-4 hrs/day)

- **Week 1-2:** CI/CD with GitHub Actions
 - Automated testing (pytest, JUnit)
 - Docker image building & pushing
 - Deployment automation
- **Week 3-4:** AWS EKS (Managed Kubernetes)
 - Setting up EKS cluster
 - Deploying containers to EKS
 - Monitoring with CloudWatch
 - Scaling & cost management

Weekends (8 hrs/day)

- **Saturday:** Build GitHub Actions pipeline
- **Sunday:** Deploy to AWS EKS cluster

Resources

- GitHub Actions docs (free)
- AWS EKS documentation (free)
- Travis CI / GitLab CI (alternatives to understand)

Monthly Project

- Full CI/CD Pipeline:
 - GitHub Actions workflow
 - Automated tests on every push
 - Docker build & push to ECR
 - Automatic deployment to EKS
 - Health checks, rollback strategies
 - Monitoring with CloudWatch

Deliverables

-  Complete CI/CD workflow in GitHub Actions

-  App automatically deployed to EKS
 -  Monitoring & alerting set up
 -  Production-ready infrastructure
-

PHASE 5: Microservices & Go (Months 13-14)

Goal: Learn Go, build microservices, scale architecture

Month 13 — Go Language Fundamentals

Weekdays (3-4 hrs/day)

- **Week 1-2: Go Basics**
 - Syntax, types, functions, packages
 - Error handling patterns (unique to Go)
 - Structs, interfaces, embedding
- **Week 3-4: Go Advanced Features**
 - Goroutines & channels (concurrency)
 - Go standard library (http, json, file I/O)
 - Testing in Go

Weekends (8 hrs/day)

- **Saturday:** Build Go CLI tools
- **Sunday:** Build simple REST API in Go (Gin framework)

Resources

- *Go: The Complete Developer's Guide* — Stephen Grider (Udemy)
- Official Go tour (free - tour.golang.org)
- Go blog & documentation (free)

Monthly Project

- Go REST API:
 - Stock ticker API or URL shortener
 - Using Gin web framework
 - JSON responses, error handling
 - Basic testing

- Dockerized

Deliverables

-  Go REST API on GitHub
 -  Gin framework mastered
 -  Goroutines & channels understood
 -  Dockerized Go service
-

Month 14 — Go Advanced + Microservices Architecture

Weekdays (3-4 hrs/day)

- **Week 1-2: Go Advanced**
 - Middleware patterns, authentication (JWT)
 - Database integration (PostgreSQL with GORM)
 - Testing patterns (mocking, unit tests)
- **Week 3-4: Microservices Patterns**
 - Service-to-service communication
 - API gateway pattern
 - Message queues (RabbitMQ or AWS SQS)
 - Service discovery basics

Weekends (8 hrs/day)

- **Saturday:** Build Go service with database & auth
- **Sunday:** Set up communication between Go & Spring Boot services

Resources

- *Microservices in Go* — Trevor Sawler (Udemy)
- Microservices patterns (free book online)

Monthly Project

- Multi-Service Architecture:
 - Go microservice (stock data service)
 - Spring Boot service (user service)
 - Communication via REST API + message queue
 - Docker Compose for local development

- Proper logging & monitoring

Deliverables

-  Two microservices communicating
 -  Docker Compose orchestration
 -  Proper error handling & resilience
 -  Architecture diagrams documented
-

PHASE 6: AI/ML Integration (Months 15-16)

Goal: Add intelligence to your apps

Month 15 — Machine Learning Fundamentals

Weekdays (3-4 hrs/day)

- **Week 1-2: ML Basics**
 - NumPy, Pandas, Matplotlib
 - Data preprocessing, exploration
 - Train/test split, cross-validation
- **Week 3-4: Supervised Learning**
 - Linear regression, logistic regression
 - Decision trees, random forests
 - Model evaluation metrics

Weekends (8 hrs/day)

- **Saturday:** Work with real datasets (Kaggle)
- **Sunday:** Build & train ML model

Resources

- *Machine Learning A-Z* — Kirill Eremenko (Udemy)
- Kaggle datasets (free)
- Scikit-learn documentation (free)

Monthly Project

- ML Model:
 - Stock price predictor OR
 - Sentiment analyzer for trading
 - Trained & tested properly
 - Model saved for deployment

Deliverables

-  Trained ML model on GitHub
 -  Jupyter notebook with full analysis
 -  Model evaluation metrics documented
-

Month 16 — Deep Learning + Deployment

Weekdays (3-4 hrs/day)

- **Week 1-2:** Deep Learning
 - TensorFlow/Keras basics
 - Neural networks, convolutional networks
 - Training loops, loss functions
- **Week 3-4:** ML Model Deployment
 - FastAPI for serving models
 - Containerizing ML services
 - Integration with web/mobile apps

Weekends (8 hrs/day)

- **Saturday:** Build deep learning model (image classification or NLP)
- **Sunday:** Deploy as API + integrate with React/Flutter

Resources

- *Deep Learning A-Z* — Kirill Eremenko (Udemy)
- FastAPI documentation (free)
- TensorFlow serving docs (free)

Monthly Project

- AI-Powered Full Stack App:
 - Deep learning model (TensorFlow)

- FastAPI backend for model serving
- React frontend calling ML API
- Flutter mobile app (optional)
- Deployed on AWS Lambda or ECS

Deliverables

-  ML service deployed
 -  Integrated with React & Flutter apps
 -  Model versioning & monitoring
 -  Performance metrics documented
-

PHASE 7: Advanced AWS + Solutions Architect Prep (Month 17)

Weekdays (3-4 hrs/day)

- **Week 1-2:** AWS Advanced Services
 - DynamoDB (NoSQL), ElastiCache (caching)
 - Step Functions (orchestration)
 - Cognito (authentication at scale)
- **Week 3-4:** AWS Solutions Architect Prep
 - Designing for resilience, performance, cost
 - Well-Architected Framework deep dive
 - Real-world architecture case studies

Weekends (8 hrs/day)

- **Saturday:** Design system architecture for your apps
- **Sunday:** AWS Solutions Architect exam prep

Resources

- *AWS Solutions Architect Associate* — Stephane Maarek (Udemy)
- AWS whitepapers (free)
- A Cloud Guru practice tests

 **Certification Target** AWS Solutions Architect Associate — prepare & take exam

Monthly Project

- Architecture Design:
 - Design scalable, resilient system
 - Document architecture (diagrams)
 - Cost estimation
 - Security best practices
 - Disaster recovery plan

Deliverables

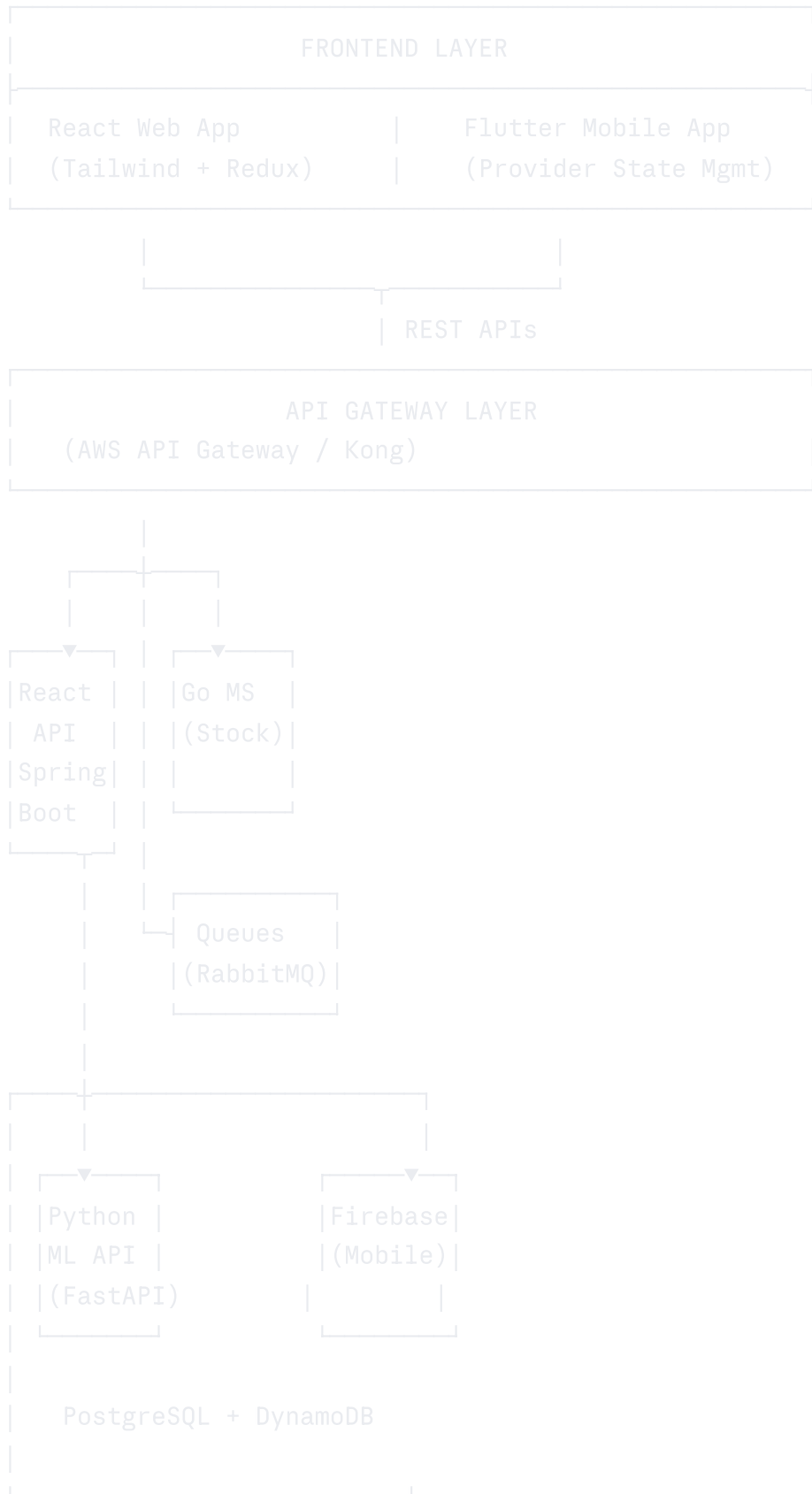
-  Architecture diagrams (draw.io or Lucidchart)
 -  AWS Solutions Architect exam preparation
 -  Understanding of well-architected principles
-

PHASE 8: Capstone Project (Month 18)

FINAL PROJECT: "SmartTrader Pro Platform"

Complete full-stack application combining ALL skills:

Architecture Overview



Infrastructure:

- └ Docker (containerization)
- └ Kubernetes (orchestration on EKS)
- └ AWS services (RDS, S3, Lambda, etc.)
- └ GitHub Actions (CI/CD)

Features to Build

User Accounts & Authentication

- JWT-based auth across all services
- Multi-factor authentication (optional)
- Role-based access control (user, trader, admin)

Trading Dashboard (Web - React)

- Real-time stock price charts (WebSockets)
- Portfolio management (buy, sell, track)
- Technical analysis indicators (RSI, MACD, Bollinger Bands)
- Watchlist functionality
- Trade history & P&L tracking
- Dark mode / light mode themes

Mobile App (Flutter)

- Same features as web (responsive)
- Push notifications for price alerts
- Offline capability (cache recent data)
- Biometric login (fingerprint/Face ID)
- Portfolio widget on home screen
- Smaller file size than web version

Microservices Backend

- **Spring Boot Service:** User management, portfolio, transactions
- **Go Service:** Real-time stock data fetching, price streaming
- **Python FastAPI Service:** ML-based stock predictions & sentiment analysis from news

AI/ML Integration

- Stock price prediction model (TensorFlow)
- Sentiment analysis on financial news (NLP)
- Anomaly detection in trading patterns
- Recommendation engine (buy/sell signals)

DevOps & Infrastructure

- Docker containers for all services

- Kubernetes deployment (EKS on AWS)
- GitHub Actions CI/CD pipeline
 - Automated tests on every commit
 - Docker builds & pushes
 - Automated deployment to EKS
- Monitoring with CloudWatch
- Database migrations with Flyway
- Secret management with AWS Secrets Manager

Database Design

- PostgreSQL: users, portfolios, transactions, watchlist
- DynamoDB: real-time stock prices (high throughput)
- Redis cache: frequently accessed data

Testing & Quality

- Unit tests: 80%+ coverage (Python, Java, Go)
- Integration tests: REST APIs
- Widget tests: Flutter
- Load testing: simulate 1000+ concurrent users

Deployment

- Frontend: S3 + CloudFront
- Mobile: Testable APK + iOS build ready (for app stores)
- Backend: EKS cluster with auto-scaling
- Database: RDS (PostgreSQL) + DynamoDB
- CI/CD: Fully automated pipeline

Documentation

- Architecture diagrams (C4 model)
 - API documentation (Swagger/OpenAPI)
 - Database schema diagrams
 - Deployment & scaling guides
 - User guide with screenshots
-

PERSONAL LEARNING — Trading (Parallel Track)

Note: This is done in your spare time (evenings/weekends), NOT part of the main 3-4 hrs/day commitment.

Learning Path

- **Month 1-2:** Stock market fundamentals
 - How stock markets work
 - Reading candlestick charts
 - Technical indicators (RSI, MACD, Bollinger Bands)
 - Fundamental analysis basics
- **Month 3-4:** Long-term investing
 - Value investing principles
 - Dividend stocks
 - Index funds, ETFs
 - Portfolio construction
 - Risk management
- **Month 5-6:** Day trading & technical analysis
 - Intraday trading patterns
 - Support & resistance levels
 - Volume analysis
 - Trading psychology
- **Ongoing:** Paper trading
 - Practice trades without real money
 - Use Alpaca, Interactive Brokers, or your broker's simulator
 - Backtest strategies
 - Track performance

Resources

- Investopedia (free)
- TradingView charts (free tier)
- Coursera — Financial Markets by Yale (free)
- YouTube — Andrei Jikh, Graham Stephan (free)
- *A Random Walk Down Wall Street* by Burton Malkiel (book)

Important Notes

⚠️ Trading Disclaimer:

- You will NOT make money in year 1 — treat it as education
- Paper trading first before using real money
- Risk management is critical
- Emotions drive 90% of trading losses
- Long-term investing statistically outperforms day trading
- Only trade with money you can afford to lose

🏆 Certifications Timeline

Month	Certification	Cost	Status
Month 10	AWS Cloud Practitioner	~\$100	Prepare
Month 11	AWS Cloud Practitioner Exam	~\$100	TAKE & PASS ✅
Month 17	AWS Solutions Architect Associate	~\$150	Prepare
Month 17	AWS Solutions Architect Exam	~\$150	TAKE & PASS ✅
Month 18	AWS Developer Associate (Optional)	~\$150	Optional

📅 Weekly Schedule Template

Day	Focus	Hours
Monday-Friday	Current phase topic + coding	3-4 hrs
Saturday	Hands-on project / building	4 hrs
Sunday	Review + LeetCode + next week prep	4 hrs
Anytime	Trading education (separate time)	1-2 hrs/week

💡 Success Principles

1. **One skill at a time** — Don't jump around. Follow the phase progression.
 2. **Projects > Tutorials** — Build real apps. This is what gets you hired.
 3. **GitHub is your resume** — Recruiters check your projects before your CV.
 4. **Test your code** — Write tests. Shows professionalism.
 5. **Deploy every project** — If it's not live, it's not real.
 6. **LeetCode consistency** — 30 minutes daily, 5 days/week minimum.
 7. **Sleep > grind** — 8 hours sleep = better learning. Don't sacrifice it.
-

🎯 End-of-18-Months Status

You Will Be Able To:

- ☒ Build full-stack web apps (React + Spring Boot)
- ☒ Build cross-platform mobile apps (Flutter iOS & Android)
- ☒ Design microservices architectures (Go + Spring Boot)
- ☒ Deploy to AWS at scale (EKS, RDS, Lambda, etc.)
- ☒ Build & deploy ML models (Python + FastAPI)
- ☒ Set up CI/CD pipelines (GitHub Actions)
- ☒ Manage infrastructure as code (Kubernetes, Docker)
- ☒ Write quality, tested code (80%+ coverage)
- ☒ Design databases (PostgreSQL + NoSQL)
- ☒ Understand trading & market fundamentals

Job-Ready Roles:

Role	Salary Range
Full Stack Engineer	\$120K-\$160K
Mobile Engineer (Flutter)	\$110K-\$150K
Backend Engineer	\$130K-\$170K
Cloud Engineer	\$140K-\$180K
Full Stack + Cloud	\$140K-\$200K 🎯

Next Steps

1. **Start with Week 1** → Deep dive into Python Decorators (we already started!)
 2. **Push everything to GitHub** → Get into the habit NOW
 3. **Build in public** → Share your learning journey
 4. **Network on LinkedIn** → Follow engineers, tech companies, mentors
 5. **Stay consistent** → 18 months of consistency > 2 months of intensity
-

Support & Resources

Every Month You'll Have:

-  Detailed day-by-day breakdown
-  Project requirements & rubric
-  Resource links (free + paid)
-  Checklist to track progress

Keep Learning:

- Udemy courses (you have subscription)
 - YouTube: Corey Schafer, Stephen Grider, Maximilian Schwarzmüller
 - Blogs: Baeldung, Medium, Dev.to
 - Official docs: Always the best source
 - Communities: Reddit r/learnprogramming, Dev.to, Discord servers
-

You've got this! 💪 Let's build something great in 18 months!

Next: Let me create this week's detailed day-by-day plan for Python Decorators!